

EXPERIMENT - 6

Develop a basic ToDo app using React Hooks (useState, useEffect) and local storage.

OBJECTIVE OF THE EXPERIMENT:

To develop a React-based ToDo application using React Hooks (useState and useEffect) that allows users to add, delete, and manage tasks while storing the task list in the browser's Local Storage so that data persists even after refreshing the page.

OUTCOME OF THE EXPERIMENT:

Students will understand:

- React Hooks (useState and useEffect).
- State management in React.
- Browser Local Storage for persistent data.
- Handling user events in React.
- Creating reusable React components.

SYSTEM REQUIREMENTS:

- Node.js and npm
- ReactJS Environment
- Visual Studio Code
- Web Browser (Chrome/Edge/Firefox)

ALGORITHM / PROCEDURE:

1. Create a new React application using Vite or Create React App.
2. Create a ToDo component.
3. Initialize task list using useState.
4. Load previously saved tasks from Local Storage using useEffect.
5. Allow users to enter a new task.
6. Add the task to the list.
7. Save updated tasks to Local Storage whenever the task list changes.
8. Display all tasks.
9. Delete a task when the Delete button is clicked.
10. Run the application and verify task persistence after page refresh.

PSEUDOCODE:

START

Initialize tasks using useState

Load tasks from Local Storage using useEffect

Display input textbox

Display Add button

IF Add button clicked THEN

 Add task to task list

ENDIF

Save updated task list to Local Storage

Display all tasks

IF Delete button clicked THEN

 Remove selected task

ENDIF

Update Local Storage

END

App.jsx

```
import React, { useState, useEffect } from "react";

function App() {

  const [task, setTask] = useState("");

  const [tasks, setTasks] = useState(() => {
    const savedTasks = localStorage.getItem("tasks");
    return savedTasks ? JSON.parse(savedTasks) : [];
  });

  useEffect(() => {
    localStorage.setItem("tasks", JSON.stringify(tasks));
  }, [tasks]);

  const addTask = () => {
    if (task.trim() === "") return;

    setTasks([...tasks, task]);
    setTask("");
  };
}
```

```

const deleteTask = (index) => {
  const updatedTasks = tasks.filter((_, i) => i !== index);
  setTasks(updatedTasks);
};

return (
  <div style={{ textAlign: "center", marginTop: "30px" }}>

    <h2>React ToDo App</h2>

    <input
      type="text"
      placeholder="Enter Task"
      value={task}
      onChange={(e) => setTask(e.target.value)}
    />

    <button onClick={addTask}>Add</button>

    <ul>
      {tasks.map((item, index) => (
        <li key={index}>
          {item}
          <button onClick={() => deleteTask(index)}>
            Delete
          </button>
        </li>
      ))}
    </ul>

  </div>
);
}

export default App;

```

Main.jsx

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

Install React

npm create vite@latest todo-app

Choose

React

JavaScript

Move into project

cd todo-app

Install dependencies

npm install

Run the project

npm run dev

RESULTS:

A functional React ToDo application was successfully developed using **useState** and **useEffect** hooks. Users can add and delete tasks, and the task list is stored in **Local Storage**, allowing it to persist even after refreshing the browser.

PRE LAB QUESTIONS:

1. What is the purpose of the **useState** hook in React?
2. What is Local Storage, and why is it used?

SAMPLE VIVA VOCE QUESTIONS:

1. What is the difference between **useState** and **useEffect**?
2. Why do we use Local Storage in a React application?
3. What is the purpose of the dependency array in **useEffect**?
4. How do you update the state in React?
5. What happens if Local Storage is not used in this application?
6. How does React re-render components when the state changes?
7. What is the purpose of the **map()** function in this program?
8. Why is the **key** attribute required while displaying lists in React?
9. What is event handling in React?
10. Explain the workflow of this ToDo application.